

EXPERIMENT -5

Student Name: Kapish Gupta

Branch: AIT-CSE (AIML)

Semester: 5

Subject Name: Database Management System

UID: 24BAI70577

Section/Group: 24AIT_KRG-1A

Subject Code: 24CSH-298

(A) A website maintains records of users who have registered on the platform and the status of confirmation requests sent to them. A confirmation request can either be successfully completed or may expire without completion.

For each registered user, calculate the confirmation rate, which is defined as:

$$\text{Confirmation Rate} = \frac{\text{Number of Successful Confirmations}}{\text{Total Confirmation Requests}}$$

If a user has never received any confirmation request, their confirmation rate should be considered **0**.

Display the user ID along with the confirmation rate rounded to **2 decimal places**. The result can be returned in any order.

Example

Suppose a user has the following confirmation records:

Action

confirmed

confirmed

timeout

Then:

Successful Confirmations = 2

Total Requests = 3

Confirmation Rate = $2 / 3 = 0.67$

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE)

pgAdmin (for PostgreSQL)

CODE SCREENSHOTS:

EXPERIMENT -5

```
SELECT S.user_id ,  
       ROUND(  
         AVG(  
           CASE  
             WHEN C.action = 'confirmed' THEN 1  
             ELSE 0  
           END  
         ) ,2  
       ) AS CONFIRMATION_RATE  
FROM Signups AS S  
LEFT JOIN Confirmations AS C  
ON S.user_id = C.user_id  
GROUP BY S.user_id
```

OUTPUT SCREENSHOTS:

Output:

```
CREATE TABLE  
CREATE TABLE  
INSERT 0 4  
INSERT 0 7
```

user_id	confirmation_rate
2	0.50
6	0.00
3	0.00
7	1.00

(4 rows)

EXPERIMENT -5

LEARNING OUTCOMES:

- ☐ **Understand SQL JOINS**
 - Use LEFT JOIN to keep all registered users, even if they have no confirmation records.
- ☐ **Use Aggregate Functions**
 - Use COUNT() to calculate:
 - Total confirmation requests
 - Successful (confirmed) confirmations
- ☐ **Conditional Aggregation**
 - Use CASE WHEN inside COUNT() or SUM() to count only successful confirmations.
- ☐ **Handle Missing Data**
 - Understand that a user with no confirmation request should have a confirmation rate of 0, not NULL.

CONCLUSION:

The problem demonstrates how SQL can be used to calculate a user's confirmation rate by combining data from multiple tables. By using LEFT JOIN, conditional aggregation, GROUP BY, COALESCE(), and ROUND(), we can correctly calculate the rate for every registered user, including users who have no confirmation requests. This strengthens understanding of SQL joins, aggregate functions, conditional counting, NULL handling, and data aggregation.

EXPERIMENT -5

Two legacy HR systems (A and B) have separate records of employee salaries. These records may overlap. Management wants to **merge these datasets** and identify **each unique employee** (by EmpID) along with their **lowest recorded salary** across both systems.

Objective

1. Combine two tables A and B.
2. Return each EmpID with their **lowest salary**, and the corresponding **Ename**.

(B) Problem Statement

A ride-booking company maintains information about trip requests and the users involved in those trips. Some users may be restricted from using the platform, while others are active participants. Each trip can either be completed successfully or cancelled by one of the participants.

For each day between **2013-10-01** and **2013-10-03**, determine the proportion of canceled trips among all valid trips. A trip should be considered valid only when both the customer and the driver associated with that trip are active users. The final result should display the date and the calculated cancellation rate rounded to two decimal places.

We need to calculate:	But only consider trips where:	Date between
Cancellation Rate	Client is NOT banned	2013-10-01
=	AND	And
	Driver is NOT banned	2013-10-03
Cancelled Trips		

Total Valid Trips		

SOFTWARE REQUIREMENTS:

Database Management System:

Oracle Database Express Edition (Oracle XE)

PostgreSQL Database

Database Administration Tool / Client Tool:

Oracle SQL Developer (for Oracle XE)

pgAdmin (for PostgreSQL)

CODE SCREENSHOTS:

EXPERIMENT -5

--SOLUTION 1:

SELECT

```
T.REQUEST_AT AS DAY,  
ROUND(  
  AVG(  
    CASE  
      WHEN T.STATUS IN  
        (  
          'cancelled_by_driver',  
          'cancelled_by_client'  
        )  
      THEN 1  
      ELSE 0  
    END  
  ),2  
)
```

FROM Trips t

JOIN Users c

ON

t.client_id = c.users_id

JOIN Users d

ON

t.driver_id = d.users_id

WHERE c.banned = 'No'

AND d.banned = 'No'

AND t.request_at BETWEEN

'2013-10-01'

AND

'2013-10-03'

GROUP BY T.REQUEST_AT;

EXPERIMENT -5

OUTPUT SCREENSHOTS:

Output:

```
CREATE TABLE
```

```
CREATE TABLE
```

```
INSERT 0 8
```

```
INSERT 0 10
```

day	round
2013-10-01	0.33
2013-10-02	0.00
2013-10-03	0.50

(3 rows)

LEARNING OUTCOMES:

After completing this experiment, I was able to:

- Understand how to use JOIN to combine data from multiple tables.
- Learn how to use the AVG() aggregate function to calculate average marks.
- Apply GROUP BY to organize data department-wise.
- Use ORDER BY to sort the results from highest to lowest average marks.
- Generate a meaningful analytical report by integrating student, department, subject, and marks data.

CONCLUSION:

The PostgreSQL query successfully generated a department-wise performance report by joining the relevant tables, calculating the average marks using the **AVG()** function, grouping the data by department, and sorting the results in descending order. This experiment